M14 # Plan-Validate-Execute Pipeline

Méthodologie Advanced

Workflow de production en 3 commandes : recherche, validation, exécution jusqu'au PR mergé

Vue d'ensemble

Plan-Validate-Execute est un workflow de production en trois commandes qui étend le Plan Mode natif : `/plan-start`, `/plan-validate`, `/plan-execute`. Chaque run alimente une boucle d'apprentissage ADR qui réduit les interruptions humaines au fil des projets. Le guide le présente comme une pratique de production Tier 2 ; installer ses commandes personnalisées avant usage.

Les 3 commandes

```
/plan-start    # PRD + équipe de recherche dynamique + ADRs
/plan-validate # Checks structurels + agents spécialistes déclenchés
/plan-execute  # Worktree + TDD + exécution parallèle + PR mergé
```

Lancer `/clear` entre chaque commande : chaque commande lit ses entrées depuis le disque (fichier de plan, ADRs), rien à garder en mémoire de contexte.

/plan-start : planification en 4 phases

Phase	Contenu
1. PRD & design	Requirements manquants/ambigus, conformité
2. Technique	Agents Explore, création d'ADRs si décision nouvelle
3. Scope	Sélection de l'équipe de recherche (tier 0 à 4)
4. Recherche	Agents en parallèle, `planning-coordinator` (Opus) synthétise

Si 3+ ADRs confirmés couvrent déjà la décision, la phase 2 s'auto-résout sans agent.

/plan-validate : validation à 2 couches

Couche 1 (structurelle, instantanée) : format du plan, ordre des tâches, cohérence avec les ADRs. Couche 2 (spécialistes, 0 à 8 agents selon le risque) : `security-reviewer`, `db-migration-reviewer`, `performance-reviewer`, entre autres, chacun déclenché par un critère précis (auth, migration de schéma, nouvelle route…).

Toute issue doit être résolue : auto-résolue via un ADR ou `PATTERNS.md` existant, ou tranchée par un humain, ce qui devient une règle pour la prochaine fois.

/plan-execute : jusqu'au PR mergé

Worktree isolé, scaffolding TDD, exécution parallèle des tâches indépendantes (commit par tâche), quality gate (tests + smoke test), PR et merge squash, nettoyage du worktree. Si le quality gate échoue : jusqu'à 3 tentatives d'auto-correction par des agents de debug dédiés, puis notification humaine.

Boucle d'apprentissage ADR

Chaque décision architecturale significative génère un ADR. 1 ADR : suivi. 2 ADR : recommandé par défaut. 3+ ADR : confirmé, auto-résolu lors des plans suivants, candidat à une promotion en règle dure dans `CLAUDE.md`. Le guide annonce environ 80% sur un projet à 20 plans, sans benchmark reproductible fourni.

Quand utiliser ce pipeline

Critère	Réponse
Feature multi-couches (API + DB + UI)	Oui
Changement sensible (auth, paiement, PII)	Oui
Typo, refactor trivial	Non, `/plan` natif suffit
Prototypage exploratoire aux besoins flous	Non

Coût

Mesurez tous les appels de planification, validation, implémentation et reprise. Les tarifs standard par million de tokens entrée/sortie sont $2/$10 pour Sonnet 5.5 et $4/$20 pour Opus 5.5 ; le nom d'une tâche ne suffit pas à déterminer le coût du pipeline.

Auto plan mode

Demander une planification via `--append-system-prompt` ne force pas le mode plan natif. Utilisez les permissions pour imposer une restriction d'outils :

```
alias claude-safe='claude --append-system-prompt \
  "$(cat ~/.claude/auto-plan-mode.txt)"'
```

Le fichier `auto-plan-mode.txt` instruit Claude de présenter un plan et d'attendre l'approbation explicite avant chaque outil. Utile pour les sessions sur des bases de code critiques ou inconnues, en dehors du pipeline à 3 commandes.